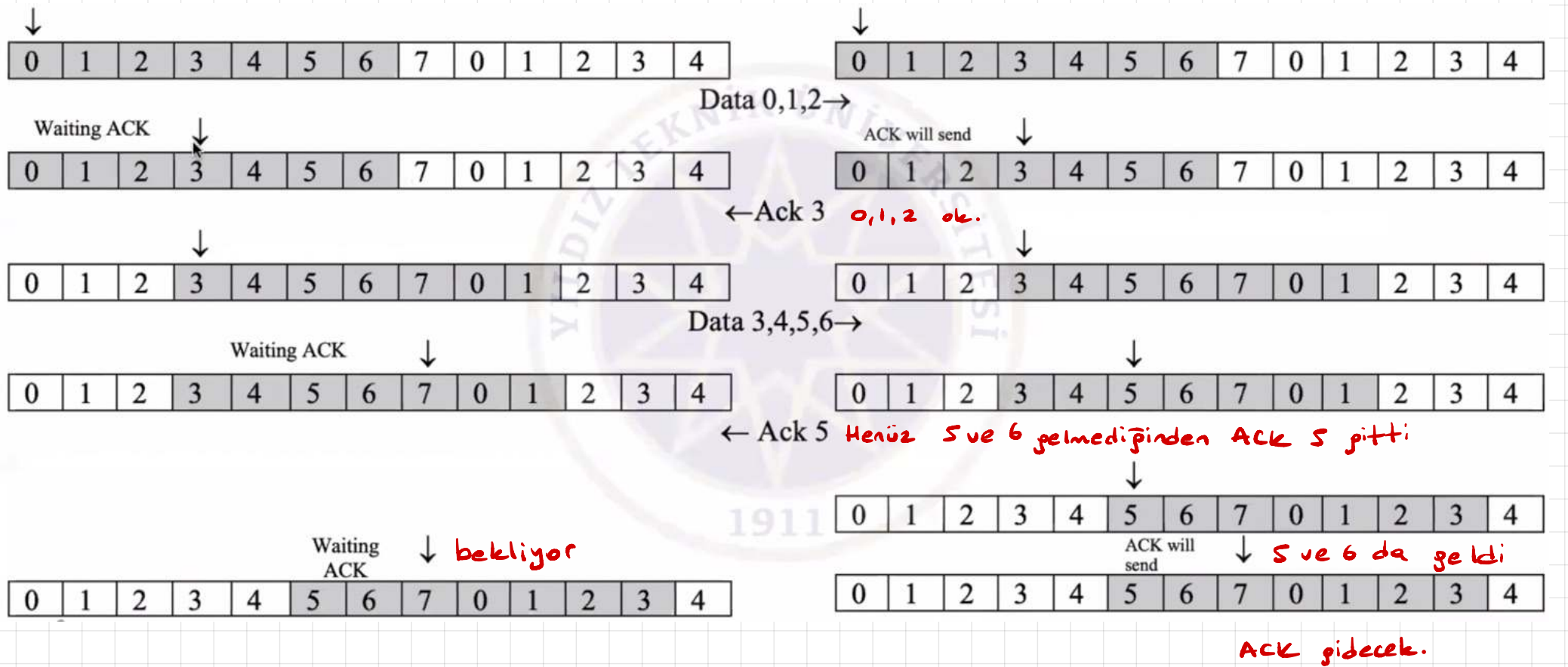


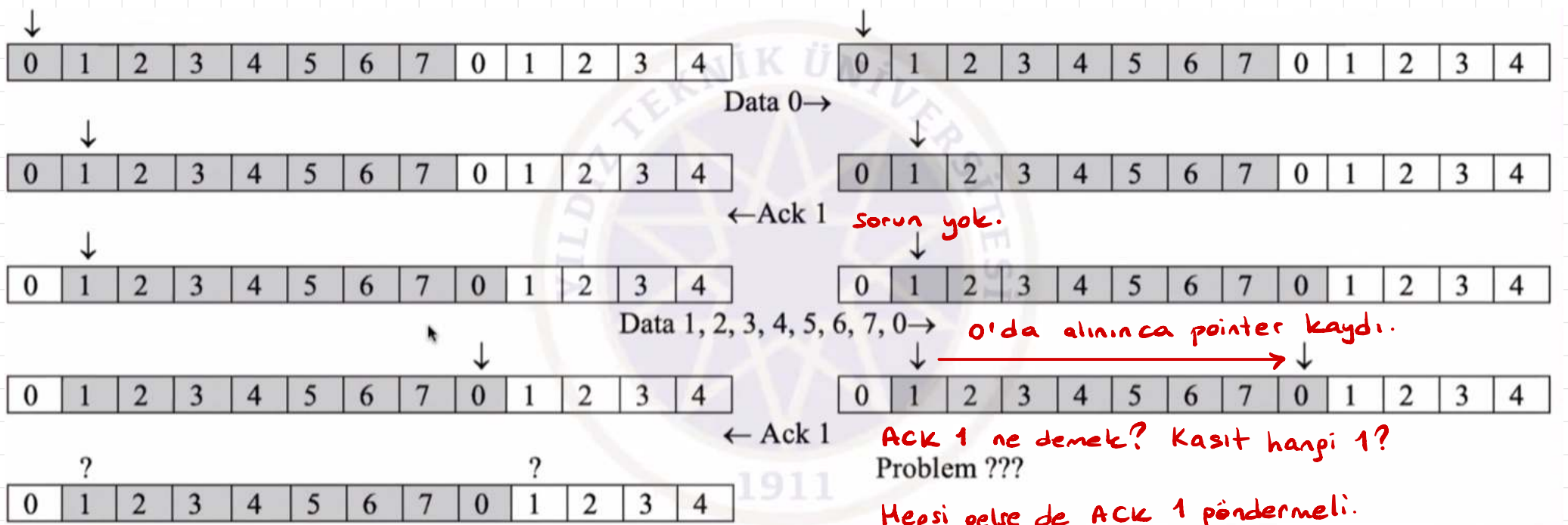
ÖRNEK



BOTH SIDE EXAMPLE



2^n olsaydı ne olurdu?



Bu nedenle son frame numarası dahil edilmez.

Sliding window Line Utilization (U) Rate

Stop and wait'te $\alpha = \frac{t_{prop}}{t_{frame}}$ demistik.

Sliding Windowsta ise $t_{frame}=1$ dolayısıyla $\alpha = t_{prop}$

Dolayısıyla

if w (window size) $\geq (2a+1)$ ise, $U = 100\%$ oluyor. (Sürekli veri pidiyor.)

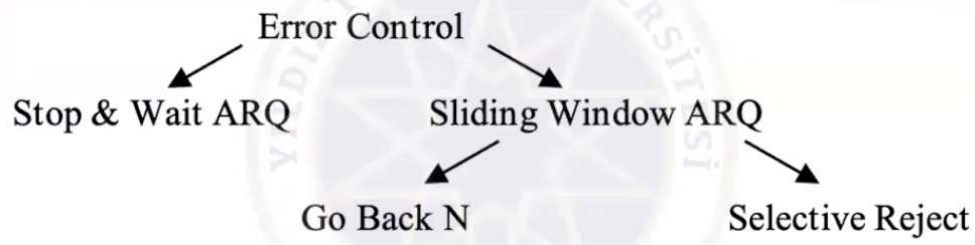
if $w < (2a+1)$ $u = \frac{w}{2a+1}$

Genellersek;

$$u = \begin{cases} 1 & w \geq 2a+1 \\ \frac{w}{2a+1} & w < 2a+1 \end{cases}$$

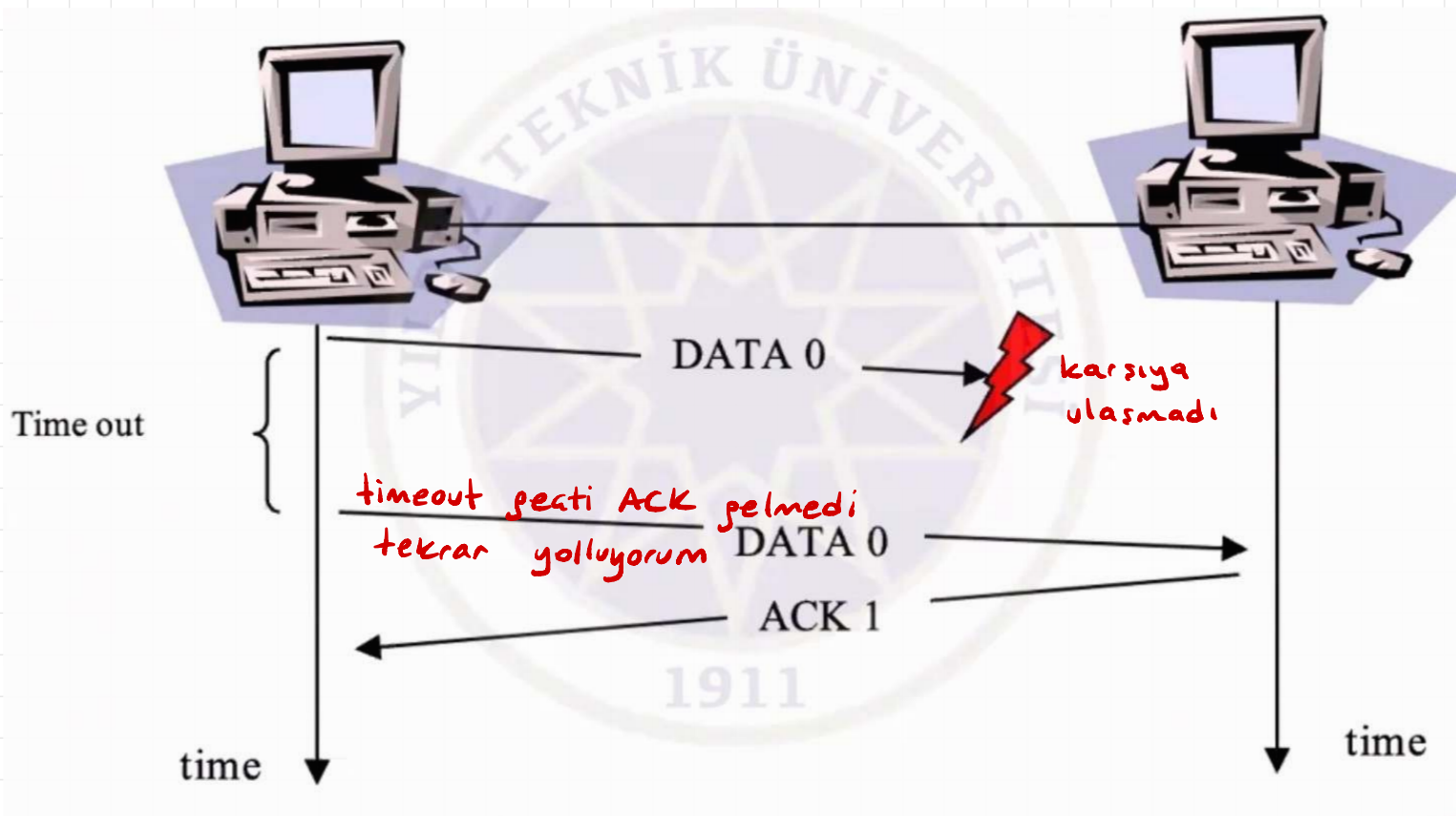
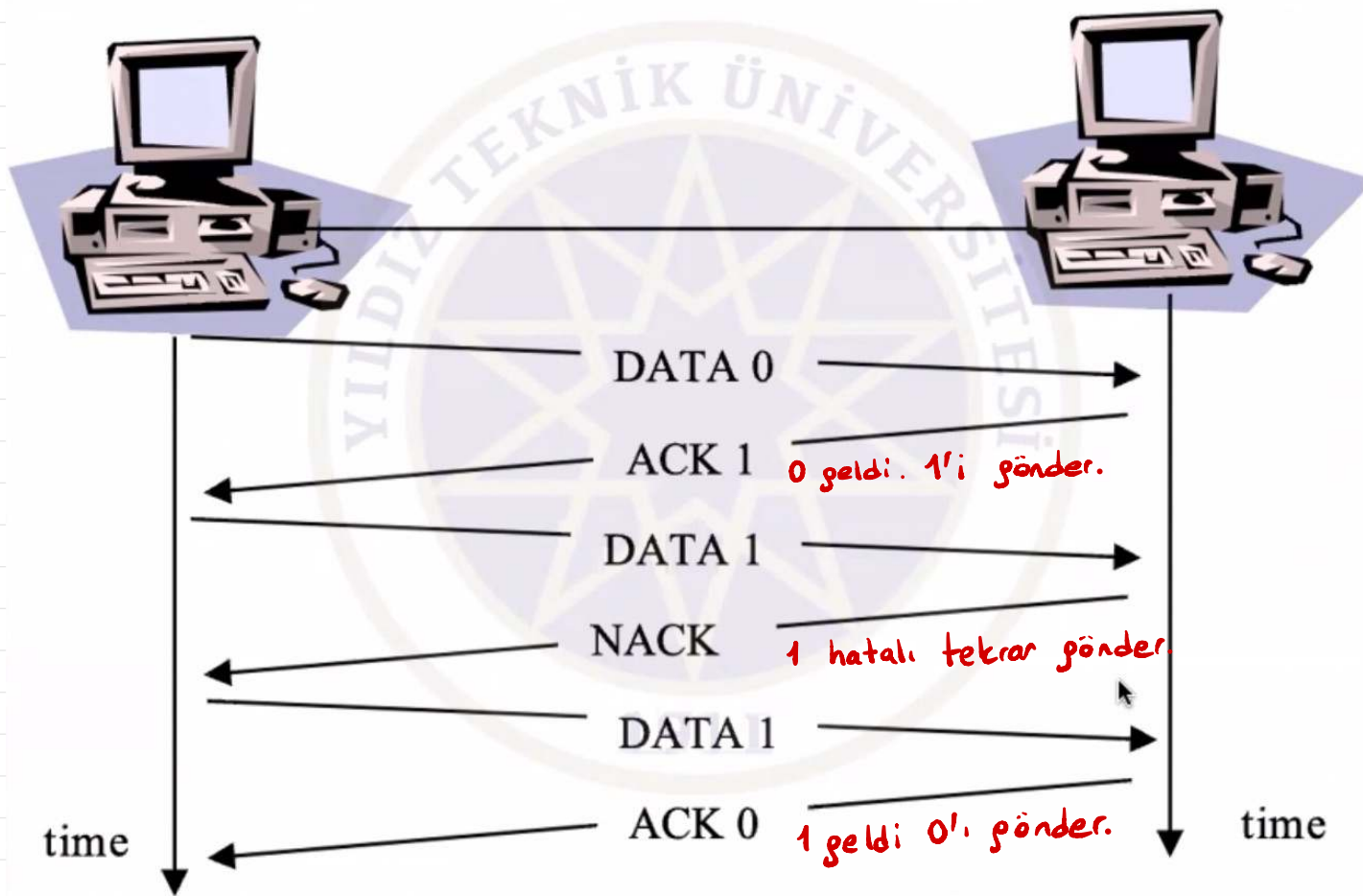
Utilization konusunda Stop and Wait'ten daha iyi.

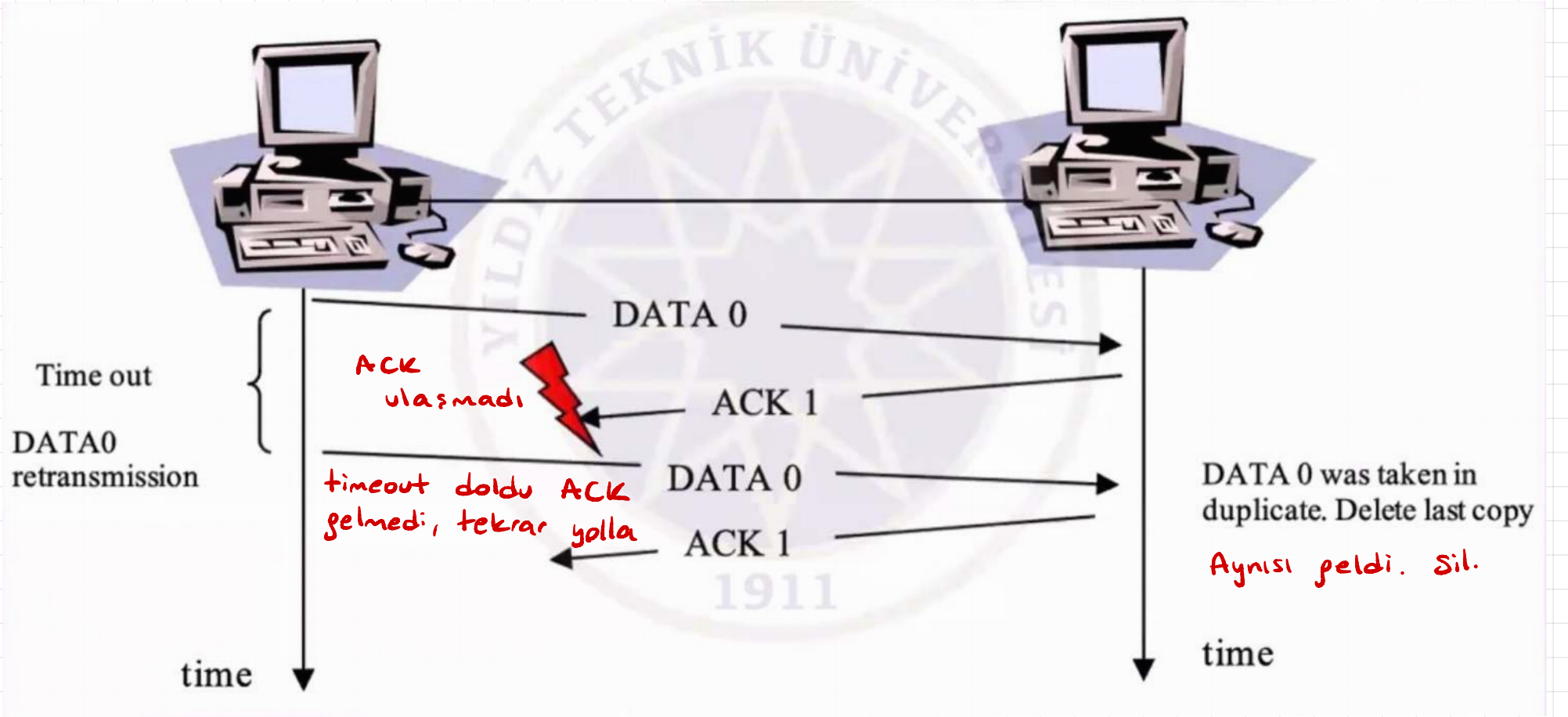
ERROR CONTROL, AUTOMATIC REPEAT REQUEST (ARQ)



Stop and Wait ARQ

Burada frame'ler ya 0 ya 1 olmak üzere 2 numaradan birini alıyor.





Sliding Window ARQ

Kontrol daha kompleks çünkü ACK almadan karşıya frame göndermeye devam ediyorum.

Aynı zamanda karşı taraf ACK verene kadar gönderdiklerimi bufferda saklıyorum.

Ancak güzelliği:

Karşı tarafa ACK-NACK gönderirken, tam olarak gelmeyen frame numarasını söyleyebiliyorum.

Receive Ready Signali:

Önce RR3 ardından RR6 gönderirsek, bu 3. 4. ve 5. frame'ler geldi, 6. bekliyorum demek.

ACK yollarken bazen toplu yolladığımız oluyordu. Ancak;

Hata bulununca, hiç beklemeden hata var mesajı yollanıyor.

Burada 2 yöntem var.

- **REJ - Reject**: Go Back N, N ve sonrasını tekrar yolla.
- **SREJ - Selective Reject**: N'i tekrar yolla.

Gönderen tarafın aynı zamanda bir Timer'ı vardır.

- **Lost Data Frame**: gönderdik hiç karşıya ulaşmadı.
- **Lost Acknowledge Frame**: gönderdik ulaştı ama bize ACK gelmedi.

Gönderen bu ikisinden hangisi olduğunu anlayamaz.

Dolayısıyla timeout olunca tekrar yollar.

GO BACK N

Frameelerin sırayla gelmesini bekler.

